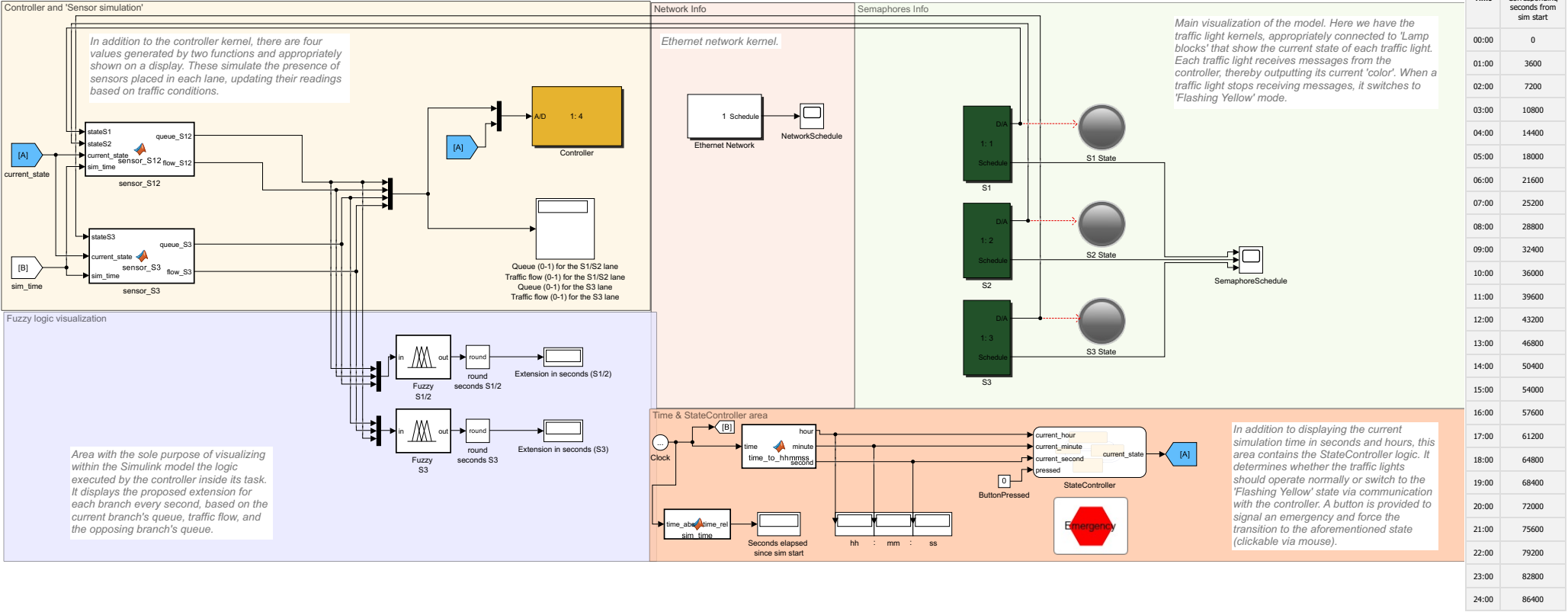




```
function [hour, minute, second] = time_to_hhmmss(time)
% time in hh:mm:ss

t = mod(time, 24*3600);

hour    = floor(t/3600);           % 0..23
minute  = floor(mod(t,3600)/60);   % 0..59
second  = floor(mod(t,60));        % 0..59
end
```

```
function time_rel = sim_time(time_abs)
% time_abs : absolute simulation time (from clock), in seconds
% time_rel : seconds elapsed since simulation start (1,2,3,...)

persistent tStart
if isempty(tStart)
    tStart = time_abs;    % start time (e.g., 43200)
end

elapsed = time_abs - tStart;    % may have decimals
elapsed = floor(elapsed);      % 0,1,2,... integers
time_rel = elapsed + 1;        % starts from 1 instead of 0
```



```

function [queue_S12, flow_S12] = sensor_S12(stateS1, stateS2, current_state, sim_time)
% sensor_S12 - Aggregated sensor for the S1/S2 branch (4 lanes)
% stateS1, stateS2: 1=red, 2=yellow, 3=green
% current_state: 1 = normal, 2 = flashing yellow
% queue_S12: normalized queue [0,1]
% flow_S12 : normalized average flow [0,1]
% sim_time: simulation time input (from clock)

% Parameters
% =====
SEED_S12 = uint32(5); % for different simulations
queue_max = 60; % full queue = 60 vehicles (4 lanes)
C_max = 3; % max vehicles that can leave in 1 s (green)
flow_max = 2.5; % maximum expected average flow (vehicles/s)
Nwin = 5; % window for average flow (5 s)
% =====

% Internal state between execution steps
persistent queue arrivalsWindow
if isempty(queue)
    queue = 25; % initial queue
end
if isempty(arrivalsWindow)
    arrivalsWindow = zeros(Nwin,1); % initial window of zeros
end

% Local RNG with seed (codegen-friendly)
% =====
persistent rngState seedUsed lastSimTime
% ALWAYS initialize persistent variables before any use
if isempty(rngState)
    rngState = uint32(1);
end
if isempty(seedUsed)
    seedUsed = uint32(0);
end
if isempty(lastSimTime)
    lastSimTime = -inf;
end
newRun = (sim_time < lastSimTime);
lastSimTime = sim_time;
if newRun || (seedUsed ~= SEED_S12)
    rngState = SEED_S12;
    if rngState == 0
        rngState = uint32(1);
    end
    seedUsed = SEED_S12;
end
% =====

% --- hourly profile ---
hour_of_day = floor(mod(sim_time/3600, 24)); % 0..23
minute_of_day = floor(mod(sim_time,3600)/60); % 0..59
second_of_day = floor(mod(sim_time,60)); % 0..59
% Generate random arrivals per lane (4 lanes)
% =====
if (hour_of_day >= 7 && hour_of_day < 9)
    p_arr_S12 = 0.30;
elseif (hour_of_day >= 9 && hour_of_day < 12)

```

```

    p_arr_S12 = 0.26;
elseif (hour_of_day >= 12 && hour_of_day < 14)
    p_arr_S12 = 0.33;
elseif (hour_of_day >= 18 && hour_of_day < 20)
    p_arr_S12 = 0.30;
elseif (hour_of_day >= 22 || hour_of_day < 6)
    p_arr_S12 = 0.02;
else
    p_arr_S12 = 0.22;
end
% arrivals = sum(rand(4,1) < p_arr_S12); % (line-item without SEED usage) @@@
arrivals = 0;
for k = 1:4
    [rngState, r] = randu_xorshift32(rngState);
    if r < p_arr_S12
        arrivals = arrivals + 1;
    end
end
% =====

% Check if S1/S2 have the green light
hasGreen = (stateS1 == 3) || (stateS2 == 3);
% Green timer - counts consecutive seconds of green light
persistent greenTimer
if isempty(greenTimer), greenTimer = 0; end

% Calculate how many cars depart
% =====
if hasGreen
    greenTimer = greenTimer + 1;
else
    greenTimer = 0;
end
if current_state == 1
    if hasGreen
        theoretical_departure = min(C_max, queue);
        [rngState, r] = randu_xorshift32(rngState);
        if greenTimer > 20
            factor = 0.75 + 0.25*r; % 0.75-1.00, mean 0.875
        else
            factor = 0.55 + 0.45*r; % 0.55-1.00, mean 0.775
        end
        departure = min(queue, round(factor * theoretical_departure));
    else
        departure = 0;
    end
elseif current_state == 2
    theoretical_departure = min(C_max, queue);
    % factor = 0.3 + 0.4*rand(); % (line-item without SEED usage) @@@
    [rngState, r] = randu_xorshift32(rngState);
    factor = 0.3 + 0.4*r; % 30-70% of maximum
    departure = min(queue, round(factor * theoretical_departure));
else
    departure = 0;
end
% =====

% Update queue
queue = queue + arrivals - departure;

```

```

if queue < 0, queue = 0; end
if queue > 80, queue = 80; end
% Update arrivals window for average flow estimation
arrivalsWindow = [arrivalsWindow(2:end); arrivals];
mean_flow = mean(arrivalsWindow); % average vehicles/s over the last Nwin seconds

% Normalize to output [0,1]
% =====
queue_S12 = queue / queue_max;
if queue_S12 > 1
    queue_S12 = 1;
end
%-----@DEBUG@-----
fprintf('%02.0f:%02.0f:%02.0f'[sensor_S12]: S1 queue = %.0f vehicles (norm = %.2f)\n', ...
    hour_of_day, minute_of_day, second_of_day, queue, queue_S12);
%-----
flow_S12 = mean_flow / flow_max;
if flow_S12 > 1
    flow_S12 = 1;
end
% =====

end
function [state, u] = randu_xorshift32(state)
% Uniform RNG [0,1) with uint32 state (xorshift32)
state = bitxor(state, bitshift(state, 13));
state = bitxor(state, bitshift(state, -17));
state = bitxor(state, bitshift(state, 5));
u = double(state) * 2.3283064365386963e-10; % 1/2^32
end

```

```

function [queue_S3, flow_S3] = sensor_S3(stateS3, current_state, sim_time)
% sensor_S3 - Aggregated sensor for the S3 branch (2 lanes)
% stateS3: 1=red, 2=yellow, 3=green
% current_state: 1 = normal, 2 = flashing yellow
% queue_S3: normalized queue [0,1]
% flow_S3 : normalized average flow [0,1]
% sim_time: simulation time in seconds (from clock)

% Parameters
% =====
SEED_S3 = uint32(6);      % for different simulations
queue_max = 40;          % full queue = 40 vehicles (2 lanes)
C_max = 2;               % max vehicles that can leave in 1 s (green)
flow_max = 1.2;          % maximum realistic flow (2 lanes, peak ~1.1 vehicles/s)
Nwin = 5;                % window for average flow (5 s)
% =====

% Internal state between execution steps
persistent queue arrivalsWindow
if isempty(queue)
    queue = 10;
end
if isempty(arrivalsWindow)
    arrivalsWindow = zeros(Nwin,1);
end

% Local RNG with seed (codegen-friendly)
% =====
persistent rngState seedUsed lastSimTime
% ALWAYS initialize persistent variables before any use
if isempty(rngState)
    rngState = uint32(1);
end
if isempty(seedUsed)
    seedUsed = uint32(0);
end
if isempty(lastSimTime)
    lastSimTime = -inf;
end
newRun = (sim_time < lastSimTime);
lastSimTime = sim_time;
if newRun || (seedUsed ~= SEED_S3)
    rngState = SEED_S3;
    if rngState == 0
        rngState = uint32(1);    % prevent null state
    end
    seedUsed = SEED_S3;
end
% =====

% --- hourly profile ---
hour_of_day = floor(mod(sim_time/3600, 24));    % 0..23
minute_of_day = floor(mod(sim_time,3600)/60);    % 0..59
second_of_day = floor(mod(sim_time,60));    % 0..59 %#ok<NASGU>
% Generate random arrivals per lane (2 lanes)
% S3 secondary branch
% =====
if (hour_of_day >= 7 && hour_of_day < 9)
    p_arr_S3 = 0.32;

```

```

elseif (hour_of_day >= 9 && hour_of_day < 12)
    p_arr_S3 = 0.28;
elseif (hour_of_day >= 12 && hour_of_day < 14)
    p_arr_S3 = 0.35;
elseif (hour_of_day >= 18 && hour_of_day < 20)
    p_arr_S3 = 0.32;
elseif (hour_of_day >= 22 || hour_of_day < 6)
    p_arr_S3 = 0.02;
else
    p_arr_S3 = 0.24;
end
% arrivals = sum(rand(2,1) < p_arr_S3); % (line-item without SEED usage) @@@
arrivals = 0;
for k = 1:2
    [rngState, r] = randu_xorshift32(rngState);
    if r < p_arr_S3
        arrivals = arrivals + 1;
    end
end
% =====

% Check if S3 has the green light
hasGreen = (stateS3 == 3);
% Green timer - counts consecutive seconds of green light
persistent greenTimer
if isempty(greenTimer), greenTimer = 0; end
% Calculate how many cars depart
% =====
if hasGreen
    greenTimer = greenTimer + 1;
else
    greenTimer = 0;
end
if current_state == 1
    if hasGreen
        theoretical_departure = min(C_max, queue);
        [rngState, r] = randu_xorshift32(rngState);
        if greenTimer > 20
            factor = 0.70 + 0.30*r; % 0.70-1.00, mean 0.850
        else
            factor = 0.50 + 0.50*r; % 0.50-1.00, mean 0.750
        end
        departure = min(queue, round(factor * theoretical_departure));
    else
        departure = 0;
    end
end
elseif current_state == 2
    theoretical_departure = min(C_max, queue); % same maximum limit
    % factor = 0.3 + 0.4*rand(); % (line-item without SEED usage) @@@
    [rngState, r] = randu_xorshift32(rngState);
    factor = 0.3 + 0.4*r; % 30-70% of maximum
    departure = min(queue, round(factor * theoretical_departure));
else
    departure = 0;
end
% =====

% Update queue
queue = queue + arrivals - departure;

```

```

if queue < 0, queue = 0; end
if queue > 60, queue = 60; end
% Update arrivals window for average flow estimation
arrivalsWindow = [arrivalsWindow(2:end); arrivals];
mean_flow = mean(arrivalsWindow);

% Normalize
% =====
queue_S3 = queue / queue_max;
if queue_S3 > 1
    queue_S3 = 1;
end
%-----@DEBUG@-----
%fprintf('%02.0f:%02.0f:%02.0f'[sensor_S3]: S3 queue = %.0f vehicles (norm = %.2f)\n', ...
%     hour_of_day, minute_of_day, second_of_day, queue, queue_S3);
%-----
flow_S3 = mean_flow / flow_max;
if flow_S3 > 1
    flow_S3 = 1;
end
% =====

end
function [state, u] = randu_xorshift32(state)
% Uniform RNG [0,1) with uint32 state (xorshift32)
state = bitxor(state, bitshift(state, 13));
state = bitxor(state, bitshift(state, -17));
state = bitxor(state, bitshift(state, 5));
u = double(state) * 2.3283064365386963e-10; % 1/2^32
end

```